

BÁO CÁO BÀI TẬP 2

1. Mục tiêu bài tập

- Tìm hiểu mô hình hoạt động của giao thức truyền tin nhẹ MQTT (Publish / Subscribe) thông qua một MQTT Broker trung gian (HiveMQ).
- Sử dụng thư viện Eclipse Paho MQTT Client để lập trình chức năng gửi (Publish) và nhận (Subscribe) dữ liệu trên môi trường Java.
- Thực hiện đóng gói thông tin cảm biến giả lập gồm DeviceName, temperature và humidity thành chuỗi định dạng JSON (sử dụng thư viện org.json) trước khi truyền đi.
- Sử dụng công cụ kiểm thử MQTT (MQTTBox hoặc console) để kiểm chứng kết quả chạy của chương trình.

2. Kiến trúc & Định dạng Dữ liệu

Kiến trúc truyền thông sử dụng một MQTT Broker công cộng (broker.hivemq.com:1883). Quy trình trao đổi tin nhắn diễn ra như sau:

1. Publisher (Luồng Gửi): Tạo dữ liệu cảm biến ngẫu nhiên hoặc nhập từ bàn phím, định dạng thành JSON byte array, và gửi lên topic `hust/iot/sensor/data` với chất lượng dịch vụ QoS = 1.
2. Subscriber (Luồng Nhận): Lắng nghe liên tục trên topic `hust/iot/sensor/data`. Khi nhận được tin nhắn, Subscriber sẽ giải mã chuỗi JSON, trích xuất các giá trị cảm biến và in ra màn hình.

Cấu trúc gói tin JSON trao đổi giữa các client:

```
{
  "DeviceName": "my-mqtt-client",
  "temperature": 30.0,
  "humidity": 60.0
}
```

3. Mã nguồn chương trình chính

3.1. Lớp `MqttPublisher.java`

```
package com.example.mqtt;

import org.eclipse.paho.client.mqttv3.MqttClient;
import org.eclipse.paho.client.mqttv3.MqttConnectOptions;
import org.eclipse.paho.client.mqttv3.MqttException;
import org.eclipse.paho.client.mqttv3.MqttMessage;
import org.eclipse.paho.client.mqttv3.persist.MemoryPersistence;
import org.json.JSONObject;

public class MqttPublisher {
    private final String brokerUrl;
```

```

private final String clientId;

public MqttPublisher(String brokerUrl, String clientId) {
    this.brokerUrl = brokerUrl;
    this.clientId = clientId;
}

public void publishData(String topic, String deviceName, double temperature, double
humidity) {
    MemoryPersistence persistence = new MemoryPersistence();
    try {
        MqttClient client = new MqttClient(brokerUrl, clientId, persistence);
        MqttConnectOptions connOpts = new MqttConnectOptions();
        connOpts.setCleanSession(true);
        connOpts.setConnectionTimeout(10);

        client.connect(connOpts);

        JSONObject payload = new JSONObject();
        payload.put("DeviceName", deviceName);
        payload.put("temperature", temperature);
        payload.put("humidity", humidity);

        MqttMessage message = new MqttMessage(payload.toString(4).getBytes());
        message.setQos(1);

        client.publish(topic, message);
        client.disconnect();
    } catch (MqttException me) {
        System.err.println("Lỗi MQTT Publisher: " + me.getMessage());
    }
}
}

```

3.2. Lớp MqttSubscriber.java

```

package com.example.mqtt;

import org.eclipse.paho.client.mqttv3.IMqttDeliveryToken;
import org.eclipse.paho.client.mqttv3.MqttCallback;
import org.eclipse.paho.client.mqttv3.MqttClient;
import org.eclipse.paho.client.mqttv3.MqttConnectOptions;
import org.eclipse.paho.client.mqttv3.MqttException;
import org.eclipse.paho.client.mqttv3.MqttMessage;
import org.eclipse.paho.client.mqttv3.persist.MemoryPersistence;
import org.json.JSONObject;
import org.json.JSONException;

public class MqttSubscriber implements MqttCallback {
    private final String brokerUrl;
    private final String clientId;
    private MqttClient client;
    private boolean isRunning = false;

    public MqttSubscriber(String brokerUrl, String clientId) {
        this.brokerUrl = brokerUrl;
        this.clientId = clientId;
    }
}

```

```

public synchronized void start(String topic) {
    try {
        client = new MqttClient(brokerUrl, clientId, new MemoryPersistence());
        MqttConnectOptions connOpts = new MqttConnectOptions();
        connOpts.setCleanSession(true);
        connOpts.setAutomaticReconnect(true);
        client.setCallback(this);
        client.connect(connOpts);
        client.subscribe(topic);
        isRunning = true;
    } catch (MqttException me) {
        System.err.println("Lỗi Subscriber: " + me.getMessage());
    }
}

public synchronized void stop() {
    try {
        if (client != null && client.isConnected()) {
            client.disconnect();
        }
    } catch (MqttException me) {
        System.err.println("Lỗi dừng Subscriber: " + me.getMessage());
    } finally { isRunning = false; }
}

public boolean isRunning() { return isRunning; }

@Override
public void connectionLost(Throwable cause) {}

@Override
public void messageArrived(String topic, MqttMessage message) {
    try {
        String payload = new String(message.getPayload());
        JSONObject json = new JSONObject(payload);
        String deviceName = json.optString("DeviceName", "Unknown");
        double temp = json.optDouble("temperature", Double.NaN);
        double hum = json.optDouble("humidity", Double.NaN);

        System.out.println("\n--- ĐÃ NHẬN GÓI TIN MQTT ---");
        System.out.println("Thiết bị: " + deviceName);
        System.out.println("Nhiệt độ: " + temp + " °C");
        System.out.println("Độ ẩm: " + hum + " %");
    } catch (JSONException e) {
        System.err.println("Lỗi định dạng JSON: " + e.getMessage());
    }
}

@Override
public void deliveryComplete(IMqttDeliveryToken token) {}
}

```

4. Kết quả chạy thử chương trình

4.1. Biên dịch và khởi chạy

```

duvx@vuxuandu:~/Documents/Hust Master/Secure Architecture for IoT Application/Exercise 2$ ./build.sh
Đang biên dịch mã nguồn Java...
Biên dịch thành công! Mã lớp nằm trong thư mục 'bin/'.
duvx@vuxuandu:~/Documents/Hust Master/Secure Architecture for IoT Application/Exercise 2$ ./run.sh
=== CHƯƠNG TRÌNH GIAO TIẾP MQTT TRÊN JAVA ===
Cấu hình mặc định:
- Broker URL : tcp://broker.hivemq.com:1883
- Topic      : hust/iot/sensor/data
-----
Bạn có muốn đổi cấu hình mặc định không? (y/n): Y
Nhập Broker URL (ví dụ: tcp://broker.hivemq.com:1883):
Nhập Topic (ví dụ: my/mqtt/topic):

===== MENU CHỨC NĂNG =====
1. Bật Subscriber (Đăng ký nhận dữ liệu)
2. Tắt Subscriber
3. Gửi dữ liệu JSON (Publish sensor data)
4. Thoát chương trình
=====
Chọn chức năng (1-4): 3

GỬI DỮ LIỆU SENSOR GIAO THỨC MQTT

```

4.2. Gửi và nhận tin nhắn MQTT

Thực hiện bật Subscriber nhận dữ liệu trên Java, đồng thời tiến hành Publish tin nhắn JSON. Subscriber bắt gói tin thành công, giải mã định dạng JSON và hiển thị đầy đủ các giá trị:

```

=====
Chọn chức năng (1-4): 3

--- GỬI DỮ LIỆU SENSOR GIAO THỨC MQTT ---
Nhập Tên thiết bị (mặc định: my-mqtt-client): my-mqtt-client
Nhập Nhiệt độ (mặc định: 30.0): 30
Nhập Độ ẩm (mặc định: 60.0): 60
[Publisher] Connecting to broker: tcp://broker.hivemq.com:1883

===== MENU CHỨC NĂNG =====
1. Bật Subscriber (Đăng ký nhận dữ liệu)
2. Tắt Subscriber
3. Gửi dữ liệu JSON (Publish sensor data)
4. Thoát chương trình
=====
Chọn chức năng (1-4): [Publisher] Connected successfully!
[Publisher] Publishing message to topic: hust/iot/sensor/data
[Publisher] Payload:
{
  "temperature": 30,
  "humidity": 60,
  "DeviceName": "my-mqtt-client"
}
[Publisher] Message published!
[Publisher] Disconnected from broker.

```